

PARADIS

Parsimonious RAnge DISpersal

User Guide — v0.1.0

Contents

1	Overview	2
2	Installation	2
3	Loading rasters	2
4	Parameter calibration	3
5	Dispersal simulation	4
6	Key API reference	6
6.1	<code>paradis.calibrate()</code>	6
6.2	<code>PopulationSimulator</code>	7
6.3	Visualisation	7
6.4	Raster imports (<code>paradis.io</code>)	8

1 Overview

PARADIS is a Python package for simulating and calibrating habitat-driven dispersal of animal populations from occurrence and habitat-suitability (HS) data. It combines:

- **Calibration** — gradient-based estimation of dispersal and growth parameters from opportunistic observations from GBIF database.
- **Simulation** — sliding-window dispersal engine based on a random-walk model, with logistic population growth.
- **GPU acceleration** — PyTorch backend with optional CUDA support for large rasters.
- **Sparse solver** — scipy SuperLU factorisation for $O(N^{1.5})$ per-window solves (vs. $O(N^3)$ dense LU).

2 Installation

```

1 # 1. Clone or install the package
2 pip install -e .
3
4 # 2. Install PyTorch with CUDA support (replace cu121 with your CUDA
   version)
5 pip install torch torchvision --index-url https://download.pytorch.org
   /whl/cu121
6
7 # 3. Install optional dependencies
8 pip install rasterio scipy geopandas scikit-image

```

CUDA check

On import `paradis` the package automatically detects GPU availability and prints a diagnostic message. If CUDA is not found a step-by-step installation guide is displayed.

3 Loading rasters

All functions accept either a NumPy array *or* a path to a `.tif` file. The raster loader handles NODATA values, integer scale/offset (e.g. uint16 ENM outputs stored as value $\times 10^{-4}$), and NaN/Inf replacement automatically.

```

1 from paradis.io import load_hs, load_obs, load_mask, read_raster
2
3 # Habitat suitability -- normalised to [0,1], NODATA and integer
4 #                       scale/offset handled automatically
5 hs = load_hs("HS_species.tif")
6
7 # Observation counts -- raw values, NODATA replaced by 0
8 obs = load_obs("Obs_species.tif")
9
10 # Binary region mask -- any non-zero pixel becomes 1
11 mask = load_mask("France_mask.tif")
12
13 # Full control via read_raster
14 data = read_raster(
15     "my_raster.tif",
16     nodata = -9999,

```

```

17     fill_value = 0.0,
18     clip_range = (0, None),
19     normalise   = True,
20 )

```

Convenience loaders

Function	Purpose	Normalisation
<code>load_hs(path)</code>	Habitat suitability	$[0, 1]$, ENM max stripped
<code>load_obs(path)</code>	Observation counts	None (raw values)
<code>load_mask(path)</code>	Binary region mask	$\{0, 1\}$
<code>read_raster(path, ...)</code>	Full control	User-defined

4 Parameter calibration

The calibration pipeline estimates four dispersal/growth parameters (E_w, n, r, T_g) and three carrying-capacity parameters (L, k, x_0) from HS and observation rasters.

Pipeline steps (executed automatically)

1. Derive current range from observations.
2. Calibrate presence threshold.
3. Estimate carrying-capacity logistic $K(HS)$.
4. Select calibration windows.
5. Compute per-window relative-abundance posteriors.
6. Fit parameters via Adam SGD.
7. Produce a summary figure.

Quick-start code

```

1  import paradis
2  from paradis.io import load_hs, load_obs, load_mask
3
4  _DATA = "examples/data/"
5
6  hs      = load_hs (_DATA + "HS_Elanus_caeruleus_Europe.tif")
7  obs     = load_obs (_DATA + "Obs_Elanus_caeruleus_Europe.tif")
8  taxa_ref = load_obs (_DATA + "Taxa_Aves_Europe.tif")
9  current_range = load_mask(_DATA + "Range_Elanus_caeruleus_Europe.tif")
10
11 results = paradis.calibrate(
12     hs, obs, taxa_ref,
13     mdd      = 21.0,
14     current_range = current_range,
15     hs_bin_width = 0.025,
16     max_iter    = 2500,
17     plot        = True,
18     species_name = "Elanus caeruleus",
19 )
20

```

```
21 print(results["Ew"], results["n"], results["r"], results["Tg"])
```

Parameters

Parameter	Description	Typical range
E_w	Expected number of random-walk steps	10^2 – 10^4
n	Risk avoidance exponent	10^2 – 10^4
r	Risk scaling exponent	10^{-5} – 10^{-2}
T_g	Characteristic growth time (years)	1–20
L	Carrying-capacity logistic ceiling	> 0
k	Logistic slope	< 0
x_0	Logistic inflection (HS value)	0–1
mdd	Prior mean dispersal distance (km)	species-specific

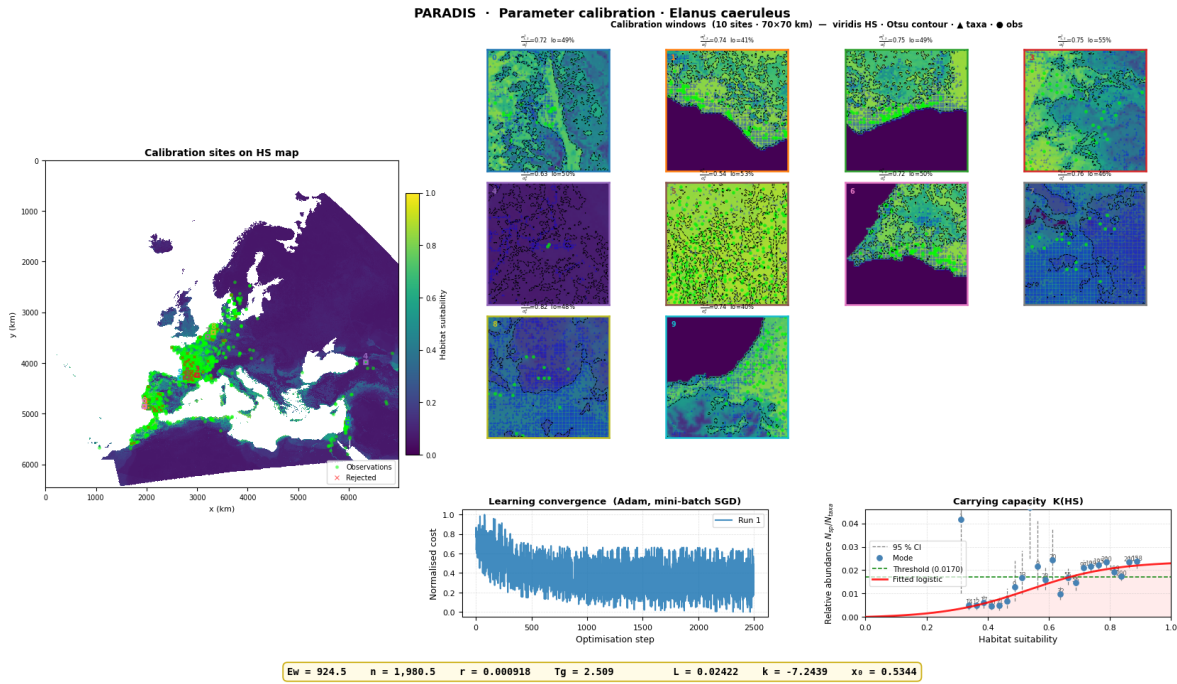


Figure 1: Calibration summary: HS map with calibration windows (left), per-window HS thumbnails (top right), learning convergence and carrying-capacity fit (bottom right). Estimated parameters shown in the banner.

5 Dispersal simulation

Given calibrated parameters, `PopulationSimulator` runs the dispersal–growth loop over a user-defined number of time steps.

Dispersal model

Each time step alternates between:

1. **Dispersal** — sliding-window solve of $M_e^\top \mathbf{d} = \mathbf{d}_0$ where $M_e = \frac{1}{1-p_f}(I - p_f W^*)$, $p_f = E_w/(1 + E_w)$, and W^* is the habitat-weighted transition matrix.
2. **Growth** — discrete logistic update $U_{t+1} = U_t + (1 - a) U_t (1 - U_t/K)$, where $a = 0.05^{1/T_g}$.

Solver options

solver	Pre-build	Per-step	Precision
'lu'	$O(N^3)$	$O(N^2)$	float16 (fast, slight loss)
'sparse'	$O(N^{1.5})$	$O(N^{1.5})$	float32 (recommended)
'gmres'	$O(N)$	$O(kN)$	float32 (iterative)

Quick-start code

```

1 from paradis.simulation import PopulationSimulator, show_expansion,
  show_final
2 from paradis.io import load_hs, load_obs, load_mask
3
4 _DATA = "examples/data/"
5
6 hs = load_hs (_DATA + "HS_Elanus_caeruleus_200km.tif")
7 france_mask = load_mask(_DATA + "depFrance_200km.tif")
8 init_distrib = load_obs (_DATA + "
  Init_distrib_Elanus_caeruleus_200km_sim.tif")
9 hs *= france_mask
10
11 sim = PopulationSimulator(
12     hs = hs,
13     ewalk = 924.5,
14     n = 1980.5,
15     r = 0.000918,
16     tgrowth = 2.509,
17     carrying_capacity_params = (0.02422, -7.2439, 0.5344),
18     solver = "sparse",
19     presence_threshold = 0.017,
20 )
21
22 sim.run(n_steps=35, init_distrib=init_distrib, window_size=60,
  sub_window=30)
23
24 show_expansion(sim)
25 show_final(sim)

```

Initial distribution

The initial distribution is a .tif raster with the same grid as the HS map. A 10×10 km founder patch (value 0.017) is provided in data/Init_distrib_Elanus_caeruleus_200km_sim.tif. It was created with:

```

1 import numpy as np, rasterio
2
3 with rasterio.open("HS_Elanus_caeruleus_200km.tif") as src:
4     transform, crs, shape = src.transform, src.crs, (src.height, src.
5         width)
6
7 init = np.zeros(shape, dtype="float32")
8 init[145:155, 85:95] = 0.017 # 10 x 10 px = 10 x 10 km
9
10 with rasterio.open("init.tif", "w", driver="GTiff",
11     height=shape[0], width=shape[1], count=1,
12     dtype="float32", crs=crs, transform=transform,
13     nodata=0.0) as dst:

```

```
12 dst.write(init, 1)
```

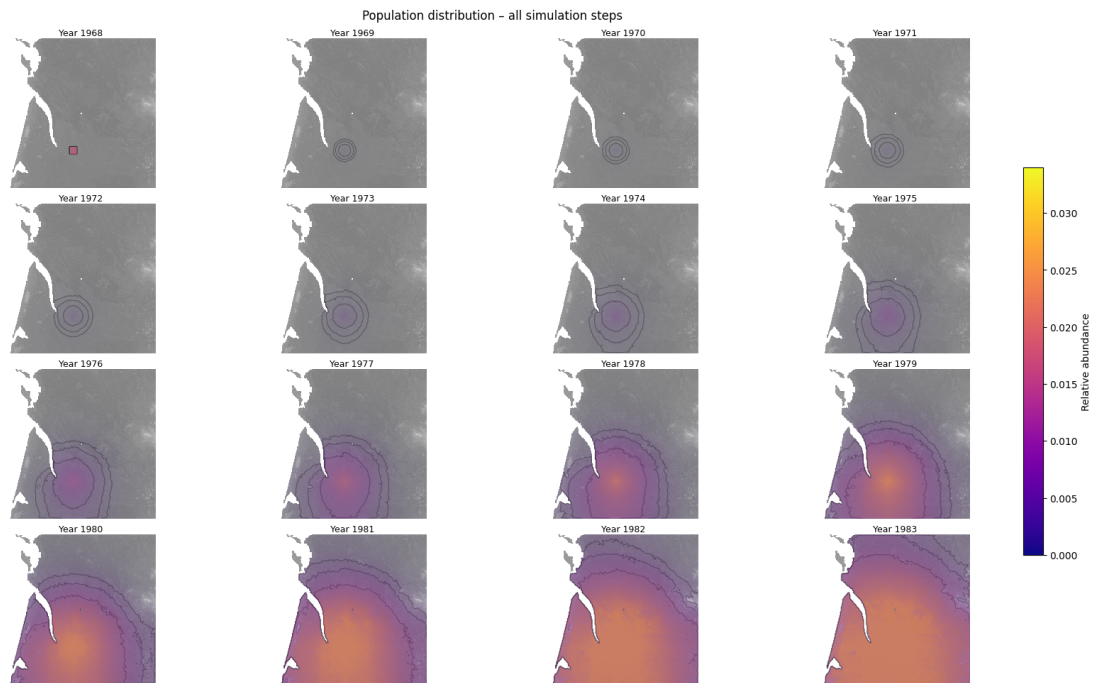


Figure 2: Simulation output for the quick start example. A small population is initialized in a $200\text{km} \times 200\text{km}$ window in Western France. Population expansion is simulated for 15 years.

6 Key API reference

6.1 `paradis.calibrate()`

```
1 results = paradis.calibrate(
2     hs,                                # habitat suitability array or .tif
3     obs,                               # observation counts array or .tif path
4     taxa_ref,                          # total species counts array or .tif
5     mdd = 21.0,                        # prior mean dispersal distance (km)
6     current_range = None,              # binary presence mask (optional)
7     valid_mask = None,                 # restrict calibration area (optional)
8     hs_bin_width = 0.025,              # width of HS histogram bins
9     max_iter = 2500,                   # number of Adam SGD steps
10    plot = True,                        # show calibration summary figure
11    save_path = None,                   # save figure to this path (optional)
12    species_name = "",                  # displayed in the figure banner
13 )
```

Runs the full seven-step calibration pipeline and returns a **dict**:

Key	Type	Meaning
Ew	float	Expected random-walk steps (dispersal reach)
n	float	Risk-avoidance exponent
r	float	Risk scaling coefficient
Tg	float	Characteristic growth time (years)
L	float	Carrying-capacity logistic ceiling
k	float	Logistic slope (negative)
x0	float	Logistic inflection point (HS value)
presence_threshold	float	Minimum density to count as present

All inputs accept either a `numpy` array or a path (`str` / `pathlib.Path`) to a `.tif` file — NO-DATA, scale/offset, and normalisation are handled automatically.

6.2 PopulationSimulator

```

1 sim = PopulationSimulator(
2     hs,                                # habitat suitability array or .
        tif path
3     ewalk,                             # Ew      expected random-walk
        steps
4     n,                                 # risk-avoidance exponent
5     r,                                 # risk scaling coefficient
6     tgrowth,                           # Tg      characteristic growth
        time (years)
7     carrying_capacity_params,          # (L, k, x0) logistic tuple
8     solver = "sparse",                 # "lu", "sparse", or "gmres"
9     presence_threshold = 0.017,        # density threshold for presence
10 )

```

Key method:

```

1 sim.run(
2     n_steps = 35,                      # number of years to simulate
3     init_distrib = init,                # initial density array (same shape as
        HS)
4     window_size = 60,                  # sliding-window diameter (pixels)
5     sub_window = 30,                  # sub-window step (pixels)
6 )

```

After `.run()`, the simulator exposes:

Attribute	Contents
<code>sim.snapshots</code>	List of density arrays, one per time step
<code>sim.hs</code>	Habitat suitability array used
<code>sim.presence_threshold</code>	Threshold used to binarise density

Solver choice. Use `solver="sparse"` (default recommendation) for grids up to $\sim 500 \times 500$ px — the SuperLU factorisation is built once per window and reused across time steps, giving $O(N^{1.5})$ scaling. `solver="lu"` (float16 dense) is fastest on GPU for small windows; `solver="gmres"` avoids factorisation entirely and is suitable when memory is the bottleneck.

6.3 Visualisation

```

1 from paradis.simulation import show_expansion, show_final
2

```

```

3 show_expansion(sim)
4 # Displays a grid of yearly distribution maps.
5 # Each panel shows the binarised presence (density > threshold)
6 # overlaid as contours on the HS background.
7
8 show_final(sim)
9 # Three-panel figure:
10 #   left      habitat suitability map
11 #   centre    initial distribution
12 #   right     final distribution after n_steps years

```

6.4 Raster imports (paradis.io)

```

1 from paradis.io import load_hs, load_obs, load_mask, read_raster

```

load_hs(path) Loads a habitat-suitability raster. Applies rasterio scale/offset, replaces NODATA and the ENM sentinel (global maximum for uint16) with 0, clips to $[0, \infty)$, then normalises to $[0, 1]$. Returns a **float32** NumPy array.

load_obs(path) Loads an observation-count or continuous-value raster. Replaces NODATA with 0 and clips negative values. Does *not* normalise — raw values are preserved so they can be used as population densities or counts. Returns a **float32** array.

load_mask(path) Loads a binary region mask. Any non-zero, non-NODATA pixel becomes 1; NODATA and zero pixels become 0. Returns a **float32** array of 0s and 1s, suitable for element-wise multiplication with HS or density arrays.

read_raster(path, band, nodata, fill_value, clip_range, normalise, drop_max_nodata)
Full-control loader used internally by the three wrappers above. Useful when none of the convenience functions match your raster's convention. All keyword arguments are optional.